TRABAJO PRÁCTICO: API REST con Persistencia en SQLite (Biblioteca)

Integración de Flask API REST, POO y Base de Datos Relacional SQLite

Objetivo del Trabajo Práctico

Reutilizar el modelo de datos y la base de datos de gestión de biblioteca desarrollada en el Trabajo Práctico de Acceso a Datos (base de datos `biblioteca.db` con tablas `autores` y `libros`), exponiendo su funcionalidad a través de una API RESTful web construida con Flask que responda a las peticiones HTTP (GET, POST, PUT, DELETE) devolviendo y recibiendo formato JSON.

Consignas y Desarrollo

Ejercicio 1: Reutilización de la Base de Datos SQLite

Utilice la estructura de base de datos SQLite `biblioteca.db` previa. Asegúrese de contar con las funciones o métodos repositorios encargados de ejecutar las sentencias SQL:

- `obtener_autores_db()` / `obtener_autor_db(id)`
- `insertar_autor_db(nombre, nacionalidad)`
- `obtener_libros_db()` / `obtener_libro_db(id)`
- `insertar_libro_db(titulo, anio, autor_id)`
- `actualizar_libro_db(id, titulo, anio, autor_id)`
- `eliminar_libro_db(id)`

Ejercicio 2: Definición e Implementación de Endpoints HTTP

Implemente una aplicación Flask (`app.py`) con las siguientes rutas/endpoints REST:

1. Recursos de Autores:
   - `GET /api/autores`: Retorna el listado completo de autores en formato JSON (Código 200).
   - `POST /api/autores`: Recibe JSON `{'nombre': str, 'nacionalidad': str}` e inserta el autor en SQLite. Retorna el nuevo registro (Código 201).

2. Recursos de Libros:
   - `GET /api/libros`: Retorna todos los libros con la información detallada de su autor vía JOIN (Código 200).
   - `GET /api/libros/<id>`: Retorna los datos de un libro específico por su ID. Si no existe, retorna código 404.
   - `POST /api/libros`: Recibe JSON `{'titulo': str, 'anio': int, 'autor_id': int}` y crea el registro en SQLite (Código 201).

- `PUT /api/libros/<id>`: Recibe JSON con los campos a modificar y actualiza el registro en SQLite (Código 200).
- `DELETE /api/libros/<id>`: Elimina el libro con la ID especificada de la base de datos (Código 200).

Ejercicio 3: Manejo de Errores y Validaciones

Asegúrese de responder con códigos HTTP e historias JSON explicativas cuando ocurran errores:
• 400 Bad Request: Cuando el cuerpo de la petición no envíe los campos obligatorios.
• 404 Not Found: Cuando se consulte, actualice o elimine un ID de libro/autor inexistente.

Ejercicio 4: Script de Prueba o Colección de Pruebas

Escriba un script secundario en Python (`test_api.py`) utilizando la librería `requests` que realice la secuencia de prueba automática:
1. Hacer GET a `/api/libros`.
2. Hacer POST para crear un autor y luego un libro.
3. Hacer PUT para actualizar el título del libro recién creado.
4. Hacer DELETE para eliminar el libro creado y verificar que la respuesta confirme la eliminación.